

AI ASSISTED CODING LAB-8

Name: D. Poornanda Chalam

Rollno: 2503a52361

Batch: 13

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

Prompt:

Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

Requirements:

- Password must have at least 8 characters.
- Must include uppercase, lowercase, digit, and special character.
- Must not contain spaces.

Buggy Code:

```
# TDD Step 1: Failed Initial Implementation / Buggy Baseline
def is_strong_password(password):
    # Bug: Only checks basic length without security criteria
    return len(password) >= 8

# Fails on missing uppercase, lowercase, digits, special characters, or spaces
# Causes: Weak passwords like "abcdefgh" or "12345678" erroneously pass validation.
```

Corrected Code & Assert Tests:

```
import string

def is_strong_password(password):
    """
    Validates password strength according to TDD security requirements:
    1. Minimum 8 characters in length.
    2. Contains at least one uppercase letter.
    3. Contains at least one lowercase letter.
    4. Contains at least one numeric digit.
    5. Contains at least one special punctuation character.
    6. Does not contain any whitespace.
    """
    if not isinstance(password, str):
        return False
    if len(password) < 8:
        return False
    if " " in password:
        return False

    has_upper = any(c.isupper() for c in password)
    has_lower = any(c.islower() for c in password)
    has_digit = any(c.isdigit() for c in password)
    has_special = any(c in string.punctuation for c in password)
```

```

    return has_upper and has_lower and has_digit and has_special

def test_password_validator():
    print("--- Running Test Assertions for Task 1 (Password Validator) ---")
    # Test Case 1: Strong valid password
    assert is_strong_password("Abcd@123") == True, "Test 1 Failed"
    print("Assertion 1 Passed: 'Abcd@123' -> Strong Password (Valid)")
    # Test Case 2: Missing uppercase and special character
    assert is_strong_password("abcd123") == False, "Test 2 Failed"
    print("Assertion 2 Passed: 'abcd123' -> Rejected (Missing uppercase & special char)")
    # Test Case 3: Missing lowercase character
    assert is_strong_password("ABCD@1234") == False, "Test 3 Failed"
    print("Assertion 3 Passed: 'ABCD@1234' -> Rejected (Missing lowercase character)")
    # Test Case 4: Contains invalid space
    assert is_strong_password("Abcd @123") == False, "Test 4 Failed"
    print("Assertion 4 Passed: 'Abcd @123' -> Rejected (Contains whitespace)")
    # Test Case 5: Strong complex password
    assert is_strong_password("SecurePass#2026") == True, "Test 5 Failed"
    print("Assertion 5 Passed: 'SecurePass#2026' -> Strong Password (Valid)")
    print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_password_validator()

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  1: pwsh  + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task1_password_validator.py
=== Task 1: Password Strength Validator (TDD) ===
TDD Requirement: Validate length (>=8), uppercase, lowercase, digit, special char, no spaces.
Applied Fix: Comprehensive validator function satisfying all security constraints.

--- Running Test Assertions for Task 1 (Password Validator) ---
Assertion 1 Passed: 'Abcd@123' -> Strong Password (Valid)
Assertion 2 Passed: 'abcd123' -> Rejected (Missing uppercase & special char)
Assertion 3 Passed: 'ABCD@1234' -> Rejected (Missing lowercase character)
Assertion 4 Passed: 'Abcd @123' -> Rejected (Contains whitespace)
Assertion 5 Passed: 'SecurePass#2026' -> Strong Password (Valid)
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code>

```

Comparison:

Password Validator Security Comparison (Naive vs TDD)	
Naive Validation (Length-Only / Buggy)	TDD Secure Validator (Full Rule Enforcement)
1. Only verifies string length >= 8 characters.	1. Enforces uppercase, lowercase, digit, and special char.
2. Accepts weak passwords without digits or special chars.	2. Strictly rejects whitespace and short strings.
3. Fails to detect insecure whitespace characters.	3. Returns deterministic Boolean status.
4. Violates OWASP password complexity guidelines.	4. Verified with 5 automated unit test assertions.

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

Prompt:

Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

Requirements:

- Classify numbers as Positive, Negative, or Zero.
- Handle invalid inputs like strings and None.
- Include boundary conditions (-1, 0, 1).

Buggy Code:

TDD Step 1: Baseline Implementation Missing Edge Case Handling

```
def classify_number(n):
    # Bug: Unhandled string/None types crash with TypeError
    if n > 0:
        return "Positive"
    elif n < 0:
        return "Negative"
    return "Zero"
```

Causes: `classify_number("hello")` and `classify_number(None)` raise `TypeError`.

Corrected Code & Assert Tests:

```
def classify_number(n):
    """
    Classifies a number as 'Positive', 'Negative', or 'Zero' using loop-based evaluation.
    Robustly handles edge cases, boundary numbers (-1, 0, 1), and invalid types (str, None, bool).
    """
    if isinstance(n, bool) or not isinstance(n, (int, float)):
        return "Invalid Input"

    rules = [
        (lambda x: x > 0, "Positive"),
        (lambda x: x < 0, "Negative"),
        (lambda x: x == 0, "Zero")
    ]

    for condition, label in rules:
        if condition(n):
            return label

    return "Invalid Input"

def test_classify_number():
    print("--- Running Test Assertions for Task 2 (Number Classifier) ---")
    # Test Case 1: Positive Number
    assert classify_number(10) == "Positive", "Test 1 Failed"
    print("Assertion 1 Passed: classify_number(10) == 'Positive'")
    # Test Case 2: Negative Number
    assert classify_number(-5) == "Negative", "Test 2 Failed"
    print("Assertion 2 Passed: classify_number(-5) == 'Negative'")
    # Test Case 3: Zero Boundary
    assert classify_number(0) == "Zero", "Test 3 Failed"
    print("Assertion 3 Passed: classify_number(0) == 'Zero'")
    # Test Case 4: Boundary conditions (-1, 1)
    assert classify_number(1) == "Positive", "Test 4 Failed"
```

```

    assert classify_number(-1) == "Negative", "Test 4 Failed"
    print("Assertion 4 Passed: Boundary values 1 and -1 verified.")
    # Test Case 5: Invalid inputs (String and None)
    assert classify_number("hello") == "Invalid Input", "Test 5 Failed"
    assert classify_number(None) == "Invalid Input", "Test 5 Failed"
    print("Assertion 5 Passed: Invalid inputs (str, None) handled gracefully.")
    print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_classify_number()

```

Output:

The screenshot shows a VS Code terminal window with the following content:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
1: pwsh  + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task2_number_classifier.py
=== Task 2: Number Classification with Loops & Edge Cases ===
TDD Requirement: Classify Positive/Negative/Zero, handle boundary (-1, 0, 1) and invalid inputs.
Applied Fix: Iterative rule evaluation with type guards for strings, None, and booleans.

--- Running Test Assertions for Task 2 (Number Classifier) ---
Assertion 1 Passed: classify_number(10) == 'Positive'
Assertion 2 Passed: classify_number(-5) == 'Negative'
Assertion 3 Passed: classify_number(0) == 'Zero'
Assertion 4 Passed: Boundary values 1 and -1 verified.
Assertion 5 Passed: Invalid inputs (str, None) handled gracefully.
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code>

```

Comparison:

Number Classification Robustness Comparison	
Basic Direct Comparison (Fragile)	TDD Loop-Based Classifier (Robust)
1. Crashes on non-numeric inputs (str, None, list).	1. Safely rejects non-numeric types with 'Invalid Input'.
2. Fails to distinguish boolean True/False from 1/0.	2. Iterates over extensible rule predicates cleanly.
3. Hardcoded chained if-else with no extensibility.	3. Accurately handles boundary conditions (-1, 0, 1).
4. Unverified behavior on boundary values (-1, 0, 1).	4. Verified with 5 automated unit test assertions.

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

Prompt:

Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

Requirements:

- Ignore case, spaces, and punctuation.
- Handle edge cases (empty strings, identical words).

Buggy Code:

```
# TDD Step 1: Naive Anagram Implementation
def is_anagram(str1, str2):
    # Bug: Case-sensitive and fails on spaces/punctuation
    return sorted(str1) == sorted(str2)

# Causes: is_anagram("Dormitory", "Dirty Room") returns False incorrectly.
```

Corrected Code & Assert Tests:

```
import re
from collections import Counter

def is_anagram(str1, str2):
    """
    Checks whether two strings are anagrams:
    1. Ignores letter casing (case-insensitive).
    2. Ignores whitespace and punctuation marks.
    3. Handles edge cases including empty strings and phrases.
    """
    if not isinstance(str1, str) or not isinstance(str2, str):
        return False

    clean1 = re.sub(r'^a-zA-Z0-9', '', str1).lower()
    clean2 = re.sub(r'^a-zA-Z0-9', '', str2).lower()

    return Counter(clean1) == Counter(clean2)

def test_anagram_checker():
    print("--- Running Test Assertions for Task 3 (Anagram Checker) ---")
    # Test Case 1: Standard single-word anagram
    assert is_anagram("listen", "silent") == True, "Test 1 Failed"
    print("Assertion 1 Passed: 'listen' and 'silent' -> True")
    # Test Case 2: Non-anagram words
    assert is_anagram("hello", "world") == False, "Test 2 Failed"
    print("Assertion 2 Passed: 'hello' and 'world' -> False")
    # Test Case 3: Phrase with differing case and whitespace
    assert is_anagram("Dormitory", "Dirty Room") == True, "Test 3 Failed"
    print("Assertion 3 Passed: 'Dormitory' and 'Dirty Room' -> True")
    # Test Case 4: Anagram with punctuation
    assert is_anagram("A gentleman!", "Elegant man") == True, "Test 4 Failed"
    print("Assertion 4 Passed: 'A gentleman!' and 'Elegant man' -> True")
    # Test Case 5: Empty strings edge case
    assert is_anagram("", "") == True, "Test 5 Failed"
    print("Assertion 5 Passed: Empty strings ('', '') -> True")
    print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_anagram_checker()
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

1: pwsh + v ^ X

```
PS C:\Users\drona\Downloads\AI Code> python task3_anagram_checker.py
=== Task 3: Anagram Checker with String Normalization ===
TDD Requirement: Ignore case, spaces, and punctuation; handle edge cases (empty strings).
Applied Fix: Regex alphanumeric normalization with Counter multiset comparison.

--- Running Test Assertions for Task 3 (Anagram Checker) ---
Assertion 1 Passed: 'listen' and 'silent' -> True
Assertion 2 Passed: 'hello' and 'world' -> False
Assertion 3 Passed: 'Dormitory' and 'Dirty Room' -> True
Assertion 4 Passed: 'A gentleman!' and 'Elegant man' -> True
Assertion 5 Passed: Empty strings ('', '') -> True
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code> 
```

Comparison:

Anagram Evaluation Comparison (Naive vs Normalized)	
Naive Direct Comparison (Fragile)	TDD Regex Normalization (Robust O(N))
1. Case-sensitive: treats 'Listen' and 'silent' as different.	1. Strips punctuation and whitespace automatically.
2. Fails when strings contain spaces or punctuation.	2. Case-insensitive comparison via lowercase normalization.
3. Inefficient sorting mechanism O(N log N).	3. Linear O(N) frequency matching via collections.Counter.
4. Throws AttributeError if inputs are non-string types.	4. Verified across phrases, punctuation, and empty strings.

Task Description #4 (Inventory Class – Apply AI to Simulate Real-World Inventory System)

Prompt:

Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

Methods:

- add_item(name, quantity)
- remove_item(name, quantity)
- get_stock(name)

Buggy Code:

```
# TDD Step 1: Unchecked Inventory Class
class Inventory:
    def __init__(self):
        self.stock = {}
    def add_item(self, name, quantity):
        self.stock[name] = quantity # Bug: Overwrites instead of adding
    def remove_item(self, name, quantity):
        self.stock[name] -= quantity # Bug: Allows negative stock & KeyError
    def get_stock(self, name):
        return self.stock[name] # Bug: Crashes if item not in stock
```

Corrected Code & Assert Tests:

```
class Inventory:
    """
    Inventory management system providing robust stock tracking:
    - add_item: increments stock for given item.
    - remove_item: safely decrements stock, preventing negative inventory.
    - get_stock: returns current stock quantity (0 for non-existent items).
    """
    def __init__(self):
        self.stock = {}

    def add_item(self, name, quantity):
        if not isinstance(name, str) or not isinstance(quantity, int) or quantity <= 0:
            return "Error: Invalid item name or quantity."
        self.stock[name] = self.stock.get(name, 0) + quantity
        return f"Added {quantity} of {name}. Current stock: {self.stock[name]}"

    def remove_item(self, name, quantity):
        if not isinstance(name, str) or not isinstance(quantity, int) or quantity <= 0:
            return "Error: Invalid quantity specified."
        current = self.stock.get(name, 0)
        if current < quantity:
            return f"Error: Insufficient stock for '{name}' (Available: {current})"
        self.stock[name] -= quantity
        return f"Removed {quantity} of {name}. Remaining stock: {self.stock[name]}"

    def get_stock(self, name):
        return self.stock.get(name, 0)

def test_inventory_system():
    print("--- Running Test Assertions for Task 4 (Inventory Class) ---")
    inv = Inventory()
    # Test Case 1: Add item and verify stock
    inv.add_item("Pen", 10)
```

```

assert inv.get_stock("Pen") == 10, "Test 1 Failed"
print("Assertion 1 Passed: Added 10 Pens -> inv.get_stock('Pen') == 10")
# Test Case 2: Remove partial quantity and verify remaining stock
inv.remove_item("Pen", 5)
assert inv.get_stock("Pen") == 5, "Test 2 Failed"
print("Assertion 2 Passed: Removed 5 Pens -> inv.get_stock('Pen') == 5")
# Test Case 3: Add second distinct item
inv.add_item("Book", 3)
assert inv.get_stock("Book") == 3, "Test 3 Failed"
print("Assertion 3 Passed: Added 3 Books -> inv.get_stock('Book') == 3")
# Test Case 4: Query non-existent item (should return 0)
assert inv.get_stock("Eraser") == 0, "Test 4 Failed"
print("Assertion 4 Passed: Non-existent item 'Eraser' -> Stock: 0")
# Test Case 5: Over-removal protection
err_msg = inv.remove_item("Pen", 10)
assert "Insufficient stock" in err_msg, "Test 5 Failed"
assert inv.get_stock("Pen") == 5, "Test 5 Failed (Stock altered)"
print("Assertion 5 Passed: Over-removal prevented safely.")
print("All 5 Assertions passed successfully!")

if __name__ == "__main__":
    test_inventory_system()

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS 1: pwsh + v ^ X

PS C:\Users\drona\Downloads\AI Code> python task4_inventory_system.py
=== Task 4: Inventory Class Simulation & Stock Control ===
TDD Requirement: Methods add_item(), remove_item(), get_stock() with stock guardrails.
Applied Fix: Encapsulated dictionary state with underflow protection and safe default lookup.

--- Running Test Assertions for Task 4 (Inventory Class) ---
Assertion 1 Passed: Added 10 Pens -> inv.get_stock('Pen') == 10
Assertion 2 Passed: Removed 5 Pens -> inv.get_stock('Pen') == 5
Assertion 3 Passed: Added 3 Books -> inv.get_stock('Book') == 3
Assertion 4 Passed: Non-existent item 'Eraser' -> Stock: 0
Assertion 5 Passed: Over-removal prevented safely.
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code>

```

Comparison:

Inventory State Management Comparison	
Unchecked Inventory Class (Unsafe)	TDD Managed Inventory Class (Robust)
1. Overwrites quantity rather than incrementing stock.	1. Properly accumulates stock via dict.get() increment.
2. Allows negative stock quantities on excess removal.	2. Protects against inventory underflow and negative numbers.
3. Crashes with KeyError on unstocked item queries.	3. Safely returns 0 for unstocked items without errors.
4. Lacks input validation for invalid negative quantities.	4. Verified across additions, removals, and edge queries.

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

Prompt:

Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

Requirements:

- Validate 'MM/DD/YYYY' format.
- Handle invalid dates.
- Convert valid dates to 'YYYY-MM-DD'.

Buggy Code:

```
# TDD Step 1: Naive Date Formatting via String Splitting
def validate_and_format_date(date_str):
    # Bug: Naive split doesn't validate calendar bounds or leap years
    parts = date_str.split("/")
    return f"{parts[2]}-{parts[0]}-{parts[1]}"

# Causes: validate_and_format_date("02/30/2023") returns "2023-02-30" (invalid date).
```

Corrected Code & Assert Tests:

```
from datetime import datetime

def validate_and_format_date(date_str):
    """
    Validates and formats a date string according to TDD requirements:
    1. Validates strict 'MM/DD/YYYY' format.
    2. Verifies real calendar validity (e.g. days per month, leap years).
    3. Converts valid dates to standard 'YYYY-MM-DD' format.
    4. Returns 'Invalid Date' for malformed or impossible dates.
    """
    if not isinstance(date_str, str):
        return "Invalid Date"

    date_str = date_str.strip()
    try:
        parsed = datetime.strptime(date_str, "%m/%d/%Y")
        return parsed.strftime("%Y-%m-%d")
    except (ValueError, TypeError):
        return "Invalid Date"

def test_date_validator():
    print("--- Running Test Assertions for Task 5 (Date Validator) ---")
    # Test Case 1: Standard valid date
    assert validate_and_format_date("10/15/2023") == "2023-10-15", "Test 1 Failed"
    print("Assertion 1 Passed: '10/15/2023' -> '2023-10-15'")
    # Test Case 2: Invalid calendar date (Feb 30th)
    assert validate_and_format_date("02/30/2023") == "Invalid Date", "Test 2 Failed"
    print("Assertion 2 Passed: '02/30/2023' -> 'Invalid Date' (Correctly caught)")
    # Test Case 3: New year date
    assert validate_and_format_date("01/01/2024") == "2024-01-01", "Test 3 Failed"
    print("Assertion 3 Passed: '01/01/2024' -> '2024-01-01'")
    # Test Case 4: Leap year date validation (Feb 29 on leap year vs non-leap year)
    assert validate_and_format_date("02/29/2024") == "2024-02-29", "Test 4 Failed (2024 Leap Year)"
    assert validate_and_format_date("02/29/2023") == "Invalid Date", "Test 4 Failed (2023 Non-Leap)"
    print("Assertion 4 Passed: Leap year rules (2024 valid vs 2023 invalid) verified.")
    # Test Case 5: Malformed string input
    assert validate_and_format_date("not-a-date") == "Invalid Date", "Test 5 Failed"
```

```
assert validate_and_format_date("13/01/2023") == "Invalid Date", "Test 5 Failed"
print("Assertion 5 Passed: Malformed strings and invalid months rejected.")
print("All 5 Assertions passed successfully!")
```

```
if __name__ == "__main__":
    test_date_validator()
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS 1: pwsh + v ^ X

```
PS C:\Users\drona\Downloads\AI Code> python task5_date_validator.py
=== Task 5: Date Validation & ISO-8601 Formatting ===
TDD Requirement: Validate MM/DD/YYYY, handle calendar errors, convert to YYYY-MM-DD.
Applied Fix: Strict datetime parsing with calendar bounds and leap year enforcement.

--- Running Test Assertions for Task 5 (Date Validator) ---
Assertion 1 Passed: '10/15/2023' -> '2023-10-15'
Assertion 2 Passed: '02/30/2023' -> 'Invalid Date' (Correctly caught)
Assertion 3 Passed: '01/01/2024' -> '2024-01-01'
Assertion 4 Passed: Leap year rules (2024 valid vs 2023 invalid) verified.
Assertion 5 Passed: Malformed strings and invalid months rejected.
All 5 Assertions passed successfully!
PS C:\Users\drona\Downloads\AI Code> 
```

Comparison:

Date Parsing & Validation Comparison	
Naive String Slicing (Unchecked)	TDD Calendar-Aware Parsing (Robust)
1. Slices strings by '/' without verifying actual dates.	1. Validates strict calendar dates via datetime.strptime.
2. Accepts non-existent dates like '02/30/2023' or '13/40/2023'.	2. Accurately handles leap years (e.g. 2024 vs 2023 Feb 29).
3. Crashes with IndexError on malformed string inputs.	3. Returns clean 'Invalid Date' on error without crashing.
4. Fails to account for leap year calendar variations.	4. Converts valid dates to standard 'YYYY-MM-DD' ISO format.